

Desenvolvimento Web 2

Aula 10

Single Page Applications com React

`l.bertholdo@ifsp.edu.br`

Conteúdo

- Passagem de Dados com props
- Atualização de Dados com state
- Método setState()
- Rotas
- Biblioteca React Router
- Exemplo – Website para Livraria
 - Alterando o Componente Navegacao
 - Alterando os Componentes Logo e Home
 - Criando os Componentes das Áreas
 - Criando os Componentes Catálogo e NotFound
 - Alterando o Componente App
 - Resultado Final

Passagem de Dados com props

- Considerando o modelo de fluxo de dados padrão do React, os dados armazenados em um componente-pai são passados para seus componentes-filhos por meio do objeto **props**.
- Como **props** é somente leitura, os componentes-filhos não podem alterar os valores de um objeto **props** recebido. Se isso for necessário, estes componentes devem fazê-lo internamente por meio de suas próprias variáveis.

*Como o fluxo de dados é unidirecional (um componente-filho não pode passar **dados alterados** para um pai), uma forma de um filho alterar dados de um pai, é fazendo com que filho **dispare** um evento no pai, e este **modifique** os dados por meio do método **setState()** do objeto **state**.*

- No exemplo a seguir, cada dado definido no componente-pai **Principal** é passado como um **atributo** para o componente-filho **Home**, que por sua vez recupera os valores de cada atributo por meio do objeto **props**.

Passagem de Dados com props

Principal.js

```
import React from "react";
import Home from "../Home";

const Principal = () => {
  const isbn = "9786589711612";
  const titulo = "Crime e Castigo";
  const autor = "Fiódor Dostoiévski";
  const tituloPagina = "Últimos lançamentos";
  return (
    <Home
      isbn={isbn}
      titulo={titulo}
      autor={autor}
      tituloPagina={tituloPagina}
    />
  );
};
export default Principal;
```

Dados definidos como variáveis, no componente-pai.

Dados passados para o componente-filho **Home** via atributos.

Home.js

```
import React from "react";

const Home = (props) => {
  console.log(props);
  return (
    <>
      <h3>{props.tituloPagina}</h3>
      <p>
        ISBN: {props.isbn}
        <br />
        Título: {props.titulo}
        <br />
        Autor: {props.autor}
      </p>
    </>
  );
};
export default Home;
```

O componente-filho recebe os dados do componente-pai em um objeto **props** como parâmetro da *arrow function*.

Últimos lançamentos

ISBN: 9786589711612
Título: Crime e Castigo
Autor: Fiódor Dostoiévski

Passagem de Dados com props

Home.js

```
const Home = (props) => {  
  console.log(props);  
  return (  
    <>  
    <h3>{props.tituloPagina}</h3>  
    <p>  
      ISBN: {props.isbn}  
      <br />  
      Título: {props.titulo}  
      <br />  
      Autor: {props.autor = "Liev Tolstói"}  
    </p>  
    </>  
  );  
};
```

Como o objeto **props** é somente leitura, ao tentar alterar o valor de um dado no componente-filho **Home**, será retornado um erro no console.

✖ ▶ Uncaught TypeError: Cannot assign to read only property 'autor' of object '#<Object>' at Home (Home.js:12:1)

No exemplo, a passagem de dados para o componente **Home** foi feita com uso de atributos, seguindo a sintaxe geral. Porém, também é possível passar dados como **conteúdo** do componente, ou seja, entre as suas *tags* de abertura de fechamento.

Esse conteúdo passado pode ser acessado no componente-filho como a propriedade nativa **children** do objeto **props**.

Principal.js

```
return (  
  <Home  
    isbn={isbn}  
    titulo={titulo}  
    autor={autor}  
  >  
    <h3>Últimos lançamentos</h3>  
  </Home>  
>);
```

Home.js

```
return (  
  <>  
    {props.children}
```

Passagem de Dados com props

Home.js

- No exemplo a seguir, os **dados dos livros** serão passados em um *array* de objetos.

O método **map()** recebe uma função *callback* que retorna cada objeto **livro** existente em **props.livros**.

```
import React from "react";

const Home = (props) => {
  console.log(props);
  return (
    <>
      <h3>{props.tituloPagina}</h3>
      {props.livros.map((livro) => (
        <p>
          ISBN: {livro.isbn}
          <br />
          Título: {livro.titulo}
          <br />
          Autor: {livro.autor}
        </p>
      ))}
    </>
  );
};

export default Home;
```

Principal.js

```
import React from "react";
import Home from "../Home";

const tituloPagina = "Últimos lançamentos";
const livros = [
  { isbn: "9786589711612", titulo: "Crime e Castigo", autor: "Fiódor Dostoiévski" },
  { isbn: "9788525056009", titulo: "Admirável Mundo Novo", autor: "Aldous Huxley" },
  { isbn: "9788535909555", titulo: "A Revolução dos Bichos", autor: "George Orwell" }
];

const Principal = () => <Home livros={livros} tituloPagina={tituloPagina} />;
export default Principal;
```

Atualização de Dados com state

- Em uma aplicação real, os dados são atualizados em resposta a alguma ação do usuário. Em React, a atualização de dados é possível graças ao seu objeto **state**.
- Por padrão, em React, somente **componentes do tipo classe** podem usar o objeto **state** para atualizar dados.
- No entanto, a versão 16.8 do React introduziu a funcionalidade **React Hooks**, que, entre outras melhorias, passou a dar suporte ao objeto **state** em **componentes funcionais**.

*Em JavaScript, **state** é um termo genérico para designar um objeto destinado a armazenar dados passíveis de serem modificados.*

Atualização de Dados com state

- Em React, cada componente possui e gerencia o seu próprio objeto **state**, ao contrário da maioria dos *frameworks* JavaScript que definem um **objeto state global** para toda a aplicação.
- O único componente que pode alterar o objeto **state** é aquele no qual o **state** foi definido, ou seja, o componente “dono” do **state**. Tentar alterar o **state** de um componente a partir de outro componente (seja pai ou filho) resulta em erro.
- Ao atualizar os dados no seu **state**, um componente-pai pode passá-los para seus componentes-filhos por meio do objeto **props**. Com isso, uma atualização no componente-pai pode ser refletida em todos os seus componentes-filhos.

Método setState()

- Nesse exemplo, primeiramente os dados são definidos por meio do objeto **state**. E, na sequência, o nome do autor é alterado.

```
import { Component } from "react";

class App extends Component {
  state = {
    isbn: "9786589711612",
    titulo: "Crime e Castigo",
    autor: "Fiódor Dostoiévski"
  }
  componentDidMount() {
    this.setState({autor: "Fiódor Mikhailovitch Dostoiévski"});
  }
  render() {
    return (
      <p>
        ISBN: {this.state.isbn} <br />
        Título: {this.state.titulo} <br />
        Autor: {this.state.autor} <br />
      </p>
    )
  }
}

export default App;
```

Para fazer esta alteração, é preciso modificar o objeto **state** por meio do método **setState()**, declarado em **Component**. Esse método recebe como parâmetro um objeto com os dados a serem alterados. Ele é chamado com **this**, que é uma referência a **Component**.

A alteração de dados deve ser declarada no método **componentDidMount()**. Esse método é chamado na sequência do método **render()** e atualiza a renderização com os dados alterados.

ISBN: 9786589711612
Título: Crime e Castigo
Autor: Fiódor Mikhailovitch Dostoiévski

Rotas

- Em aplicações React, as **rotas** são responsáveis por fazer a **navegação via links** funcionar. Para implementá-las, será utilizada a biblioteca **React Router** do React.

Para exemplificar a implementação de rotas, será retomado o desenvolvimento do website da Livraria da Subida, cujo menu de navegação não está funcional. Nesta etapa, também serão construídas as demais páginas que serão acessadas pelos links do menu de navegação.



Biblioteca React Router

- A biblioteca **React Router** oferece os seguintes componentes primários para projetar rotas: **BrowserRouter**, **Routes**, **Route**, **Link** e **Navlink**.

Alguns exemplos de sintaxe para importá-los:

Esta sintaxe encurta o nome nativo da funcionalidade **BrowserRouter** para o *alias* **Router**. Com isso, é possível simplificar a escrita do código, declarando `<Router></Router>` em vez de `<BrowserRouter></BrowserRouter>`.

```
import { BrowserRouter as Router } from "react-router-dom";  
import { Routes, Route } from "react-router-dom";  
import { Link } from "react-router-dom";  
import { NavLink } from "react-router-dom";  
// ou qualquer outra combinação de funcionalidades separadas por vírgula
```

- Os trechos de código que usam as funcionalidades desta biblioteca devem ser sempre agrupados no elemento `<BrowserRouter></BrowserRouter>` ou no seu *alias* (se houver um).
- Isso permite que esse container disponibilize as funcionalidades da biblioteca **React Router** para todos os seus componentes descendentes.

Biblioteca React Router

- Geralmente, o container **<BrowserRouter>**, ou seu **alias**, é colocado no início do **return** do componente-pai **App**.
- Isso garante que qualquer código da aplicação que use a biblioteca **React Router** tenha acesso as suas funcionalidades.
- Todas as funcionalidades da biblioteca **React Router** mencionadas são elementos de marcação JSX, ou seja, são elementos com *tags* de abertura e fechamento, conforme mostrado a seguir:

```
<BrowserRouter></BrowserRouter>  
<Routes></Routes>  
<Route></Route>  
<Link></Link>  
<NavLink></NavLink>
```

Alterando o Componente Navegacao

```
import React from "react";
import { NavLink } from "react-router-dom";
```

import do elemento `<NavLink>` da biblioteca `react-router-dom`.

```
const Navegacao = () => (
```

O elemento `<NavLink>` é convertido para a tag `<a>` da marcação HTML.

```
  <ul>
```

```
    <li>
```

```
      <NavLink to="/">Home</NavLink>
```

```
    </li>
```

```
    <li>
```

```
      <NavLink to="/historia">História</NavLink>
```

```
    </li>
```

```
    <li>
```

```
      <NavLink to="/ciencia">Ciência</NavLink>
```

```
    </li>
```

```
    <li>
```

```
      <NavLink to="/literatura">Literatura</NavLink>
```

```
    </li>
```

```
    <li>
```

```
      <NavLink to="/catalogo">Catálogo</NavLink>
```

```
    </li>
```

```
  </ul>
```

```
);
```

```
export default Navegacao;
```

O atributo `to` é convertido para o atributo `href` da tag `<a>` do HTML.

Se necessário, faça a instalação da biblioteca no projeto executando o comando `npm install react-router-dom`. Após a instalação, o arquivo `package.json` do projeto é atualizado.

```
{
  "name": "livraria",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@testing-library/dom": "^10.4.0",
    "@testing-library/jest-dom": "^6.6.3",
    "@testing-library/react": "^16.3.0",
    "@testing-library/user-event": "^13.5.0",
    "react": "^19.1.0",
    "react-dom": "^19.1.0",
    "react-router-dom": "^7.6.0",
```

Alterando o Componente Navegacao

O valor do atributo **to** é uma **barra** seguida pelo **nome de uma página** da aplicação, exceto para a **Home Page** que recebe, obrigatoriamente, apenas a barra. O nome da página admite parâmetros e também pode ser chamado de **slug** da URL.

```
<li>
  <NavLink to="/">Home</NavLink>
</li>
<li>
  <NavLink to="/historia">História</NavLink>
</li>
<li>
  <NavLink to="/ciencia">Ciência</NavLink>
</li>
<li>
  <NavLink to="/literatura">Literatura</NavLink>
</li>
<li>
  <NavLink to="/catalogo">Catálogo</NavLink>
</li>
```

Ao executar a aplicação, o navegador retornará uma mensagem relacionada à falta do container **<Router>**. Esse erro permanecerá até à última alteração a ser feita no arquivo **App.js**.

```
useLocation() may be used only in the context of a <Router> component.
```

Alterando os Componentes Logo e Home

Logo.js

```
import React from "react";
import { Link } from "react-router-dom";

const Logo = () => (
  <Link to="/">
    <h1 className="logo">
      <span>Livraria da Subida</span>
    </h1>
  </Link>
);
export default Logo;
```

O elemento **<Link>** é convertido para a tag **<a>** da marcação HTML, que direciona o usuário para a página **Home**. Como este link não pertence ao menu de navegação, usa-se o elemento **<Link>** em vez de **<NavLink>**.

Home.js

```
import React from "react";

const Home = () => (
  <main className="principal">
    <h2>Lançamentos</h2>
    <div className="card">
      <p>Conteúdos da Home Page</p>
    </div>
  </main>
);
export default Home;
```

Substitua *tag* vazia pela tag **<main>** e remova os dois livros inseridos na página **Home**.

Criando os Componentes das Áreas

- Crie estes componentes dentro da pasta **src/components** do projeto:

```
import React from "react";      Historia.js

const Historia = () => (
  <main className="principal">
    <h2>Categoria História</h2>
    <div className="card">
      <p>Conteúdos da página História</p>
    </div>
  </main>
);
export default Historia;
```

```
import React from "react";      Ciencia.js

const Ciencia = () => (
  <main className="principal">
    <h2>Categoria Ciência</h2>
    <div className="card">
      <p>Conteúdos da página Ciência</p>
    </div>
  </main>
);
export default Ciencia;
```

```
import React from "react";      Literatura.js

const Literatura = () => (
  <main className="principal">
    <h2>Categoria Literatura</h2>
    <div className="card">
      <p>Conteúdos da página Literatura</p>
    </div>
  </main>
);
export default Literatura;
```


Criando os Componentes Catálogo e NotFound

- Crie estes componentes dentro da pasta **src/components** do projeto:

Catalogo.js

```
import React from "react";

const Catalogo = () => (
  <main className="principal">
    <h2>Catálogo de livros</h2>
    <ul>
      <li>
        <a href="#">Livro 1</a>
      </li>
      <li>
        <a href="#">Livro 2</a>
      </li>
    </ul>
  </main>
);

export default Catalogo;
```

NotFound.js

```
import React from "react";
import { Link } from "react-router-dom";

const NotFound = () => (
  <main className="principal">
    <h2>404!</h2>
    <p>
      Página não encontrada. <Link to="/">Home Page</Link>
    </p>
  </main>
);

export default NotFound;
```

Em React, é comum definir um componente denominado **NotFound**, para se criar uma página de erro 404 personalizada.

Alterando o Componente App

Versão anterior

```
const App = () => {  
  return (  
    <>  
      <Topo />  
      <Principal />  
      <Rodape />  
    </>  
  );  
}
```

App.js

Importação dos componentes necessários à renderização das páginas da aplicação.

O componente **Router** deve ser o container geral da aplicação.

O componente **Topo** é o primeiro a ser inserido na marcação HTML.

Routes é um container para os componentes **Route**. Ele substitui o componente **Principal**.

O componente **Route** define os caminhos das rotas e o componente correspondente que deve ser renderizado.

O atributo **element** é usado para especificar o componente que deve ser renderizado quando o caminho da rota (**path**) for acionado.

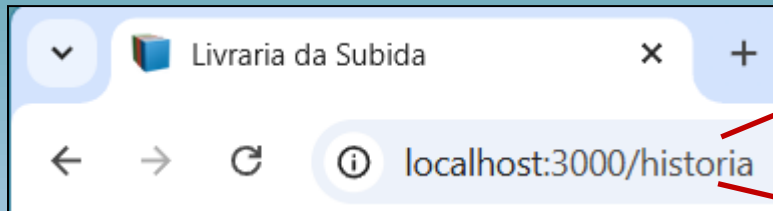
O **path** "*" é uma referência para qualquer URL indefinida, sendo apropriado para direcionar o usuário a uma página de erro 404.

holdo@ifsp.edu.br

```
import React, { Component } from "react";  
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";  
import Topo from "../components/Topo";  
import Home from "../components/Home";  
import Historia from "../components/Historia";  
import Ciencia from "../components/Ciencia";  
import Literatura from "../components/Literatura";  
import Catalogo from "../components/Catalogo";  
import NotFound from "../components/NotFound";  
import Rodape from "../components/Rodape";  
  
class App extends Component {  
  render() {  
    return (  
      <Router>  
        <Topo />  
        <Routes>  
          <Route path="/" element={<Home />} />  
          <Route path="/historia" element={<Historia />} />  
          <Route path="/ciencia" element={<Ciencia />} />  
          <Route path="/literatura" element={<Literatura />} />  
          <Route path="/catalogo" element={<Catalogo />} />  
          <Route path="*" element={<NotFound />} />  
        </Routes>  
        <Rodape />  
      </Router>  
    );  
  }  
}  
export default App;
```

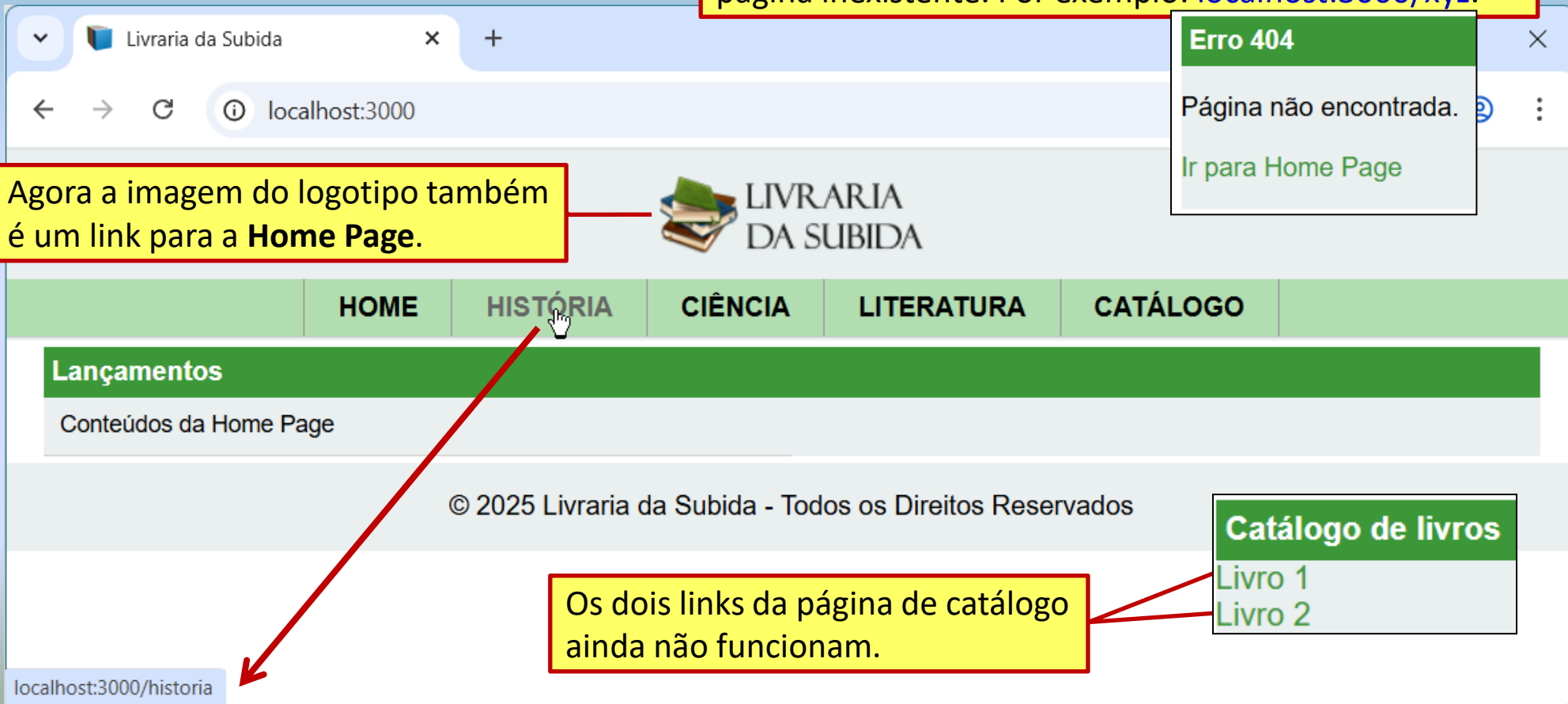
Resultado Final

O resultado final é um website dinâmico baseado em componentes React, com abas de navegação funcionais.



Ao navegar pelas páginas, observe na barra de endereços o padrão `localhost:3000/nome-da-pagina/`.

Para visualizar a página de erro 404, tente acessar uma página inexistente. Por exemplo: `localhost:3000/xyz`.



Erro 404
Página não encontrada.
[Ir para Home Page](#)

Catálogo de livros
Livro 1
Livro 2

Os dois links da página de catálogo ainda não funcionam.

Referências

- SILVA, Maurício S. **React: Aprenda Praticando**. São Paulo: Editora Novatec, 2021.
- TORRES, Fernando E. **Single Page Application**. São Paulo: Editora Senac, 2023.